Design Plan for Package R: 'BFRdist'

Daniel Luo and Jennifer Hanna

2026-08-29

1. Introduction and Motivation

Continuous data supported strictly on the unit interval, such as the Generalized Radar Vegetation Index (GRVI), are ubiquitous in polarimetric Synthetic Aperture Radar (SAR) analysis. Historically, practitioners have relied on the Beta or Kumaraswamy distributions to model these bounded features. However, as established by @bhattacharya2024, these legacy models force a trade-off between analytical tractability and morphological flexibility. The novel BFR distribution resolves this tension, offering a flexible two-parameter (a, β) model with closed-form cumulative distribution and quantile functions.

The primary motivation for developing the `BFRdist` R package for our DATA501 project was to translate this complex mathematical framework into an accessible, computationally efficient software tool. Our package implements the core distribution functions, rapid parameter estimation, and the novel quantile-based distributional diagram ($SK_G \times KU_M$) proposed by the original authors [@bhattacharya2024]. This report outlines our development architecture, addresses the reproducibility of our software following the badging framework of @frery2020, and offers a critical reflection on our development cycle.

2. Development Process and Architecture

The development of `BFRdist` was executed in structured phases, prioritizing numerical stability and object-oriented design. We strictly adhered to R package development standards, utilizing `devtools`, `roxygen2`, and `testthat`.

2.1 Core Functions and S3 Implementation

We began by implementing the standard d/p/q/r family of functions for the BFR distribution in base R. The mathematical definitions required careful handling of limits, particularly for boundary conditions approaching 0 and 1, to prevent floating-point underflow. To leverage R's object-oriented capabilities, we designed the primary estimation function, `fit_bfr()`, to return an S3 object of class `bfr`.

As illustrated in the data flow architecture above, we then developed dedicated S3 methods (`print`, `summary`, `plot`, and `predict`) to seamlessly handle this object, ensuring the user experience matches that of native R statistical functions and routes the raw data directly into the appropriate diagnostic outputs.

2.2 C++ Integration for Quantile Estimation

While Maximum Likelihood Estimation (MLE) for the BFR model requires iterative numerical optimization, @bhattacharya2024 derived a highly efficient Quantile Estimator (QE) using the first and third sample quartiles. Given that remote sensing datasets often involve wall-to-wall, multi-temporal image stacks containing millions of pixels, estimation speed is critical.

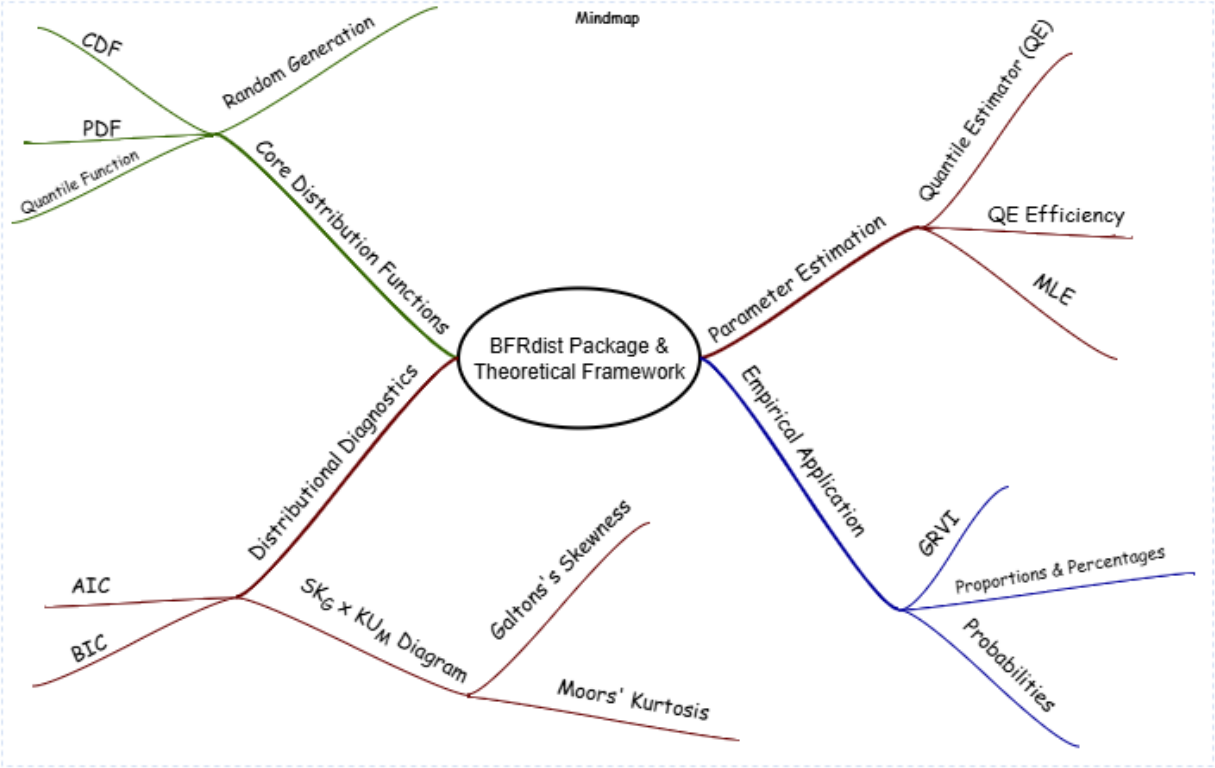


Figure 1: BFRdist Package Architecture Mindmap

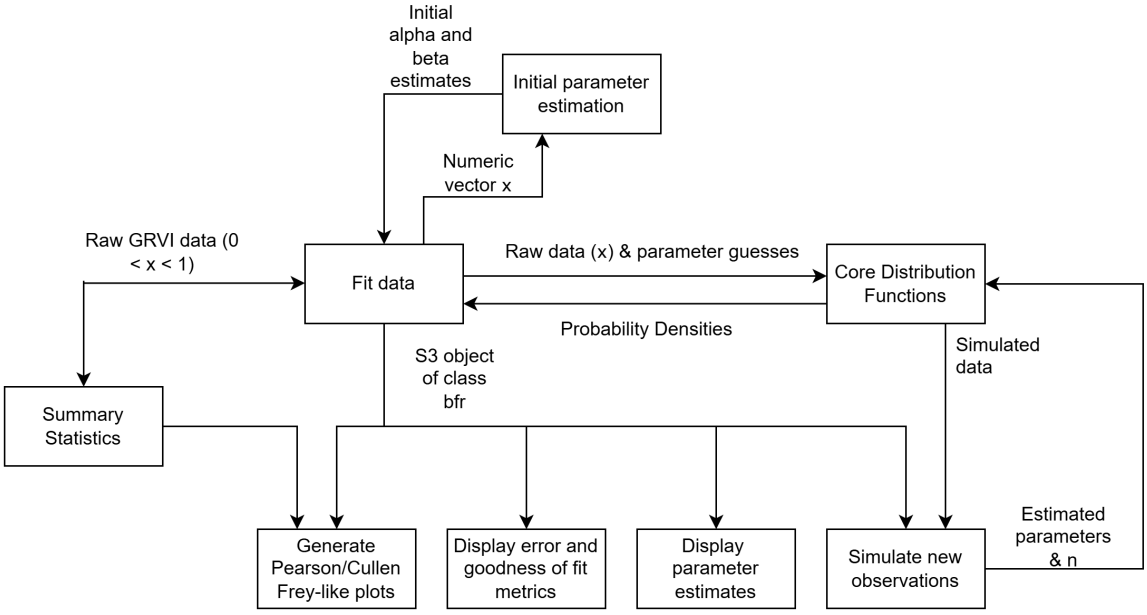


Figure 2: BFRdist Data Flow Architecture

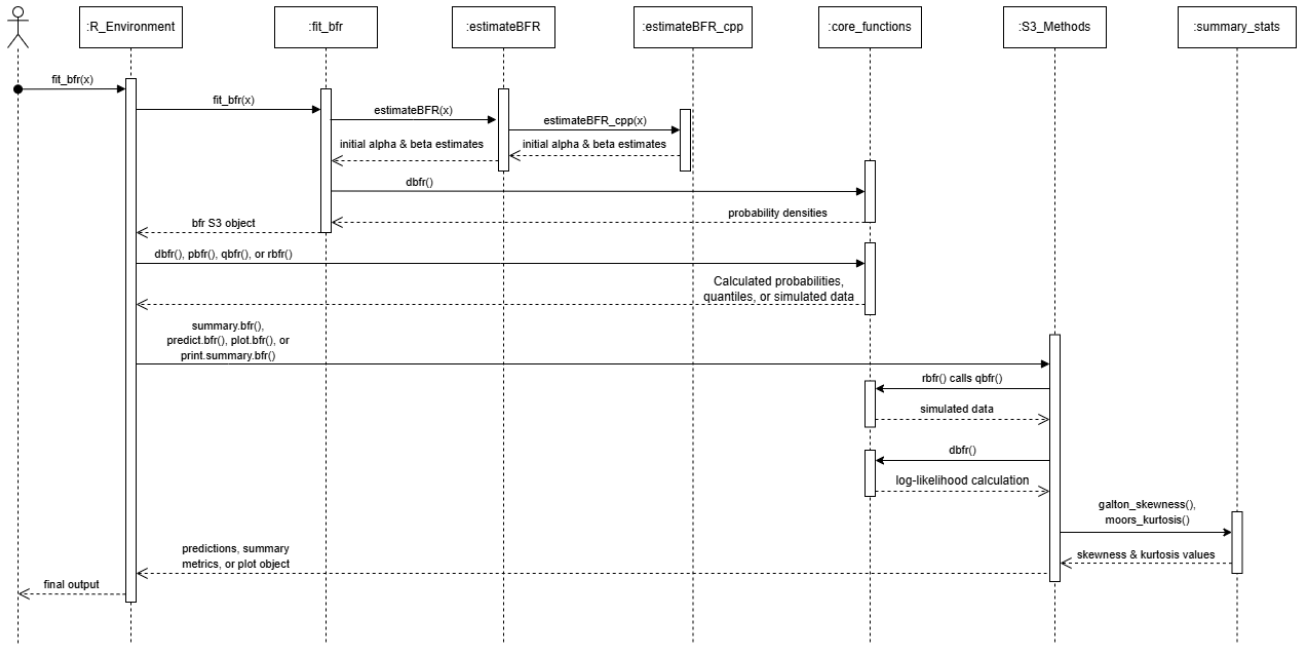


Figure 3: BFRdist Execution Sequence Diagram

To address this, we integrated C++ into our package using `Rcpp`. As shown in the execution sequence diagram, we implemented the `estimateBFR_cpp()` function, which hardcodes R's default Type 7 continuous linear interpolation algorithm to find exact quartile boundaries. By pushing the data sorting, validation, and interpolation loops to a compiled language, we achieved significant performance gains over a pure R implementation. This C++ backend seamlessly interfaces with the `fit_bfr(..., method = "qe")` frontend.

3. Reproducibility and Replicability Framework

Scientific software must be verifiable. We evaluated the architecture and distribution of `BFRdist` against the reproducibility and replicability badging criteria proposed by @frery2020.

- **R1: Data Availability:** While the package is designed to ingest proprietary polarimetric SAR data (such as empirical GRVI matrices), we engineered the `rbfr()` function to allow users to generate reproducible synthetic samples. This ensures that all examples, vignettes, and tests can be executed independently without requiring access to restricted satellite data.
- **R2 & R3: Software and Environment Availability:** The package is distributed as full source code via a public GitHub repository. It passes standard `R CMD check` without warnings or errors, ensuring cross-platform compatibility (Windows, macOS, Linux). We explicitly documented all dependencies, particularly the requirement for an active C++ compiler (Rtools/Xcode) to build the `Rcpp` components from source. The documentation strictly separates functional mechanics (handled in the vignette) from source architecture.
- **R4: Reproducible Workflow:** Our comprehensive testing suite, utilizing `testthat`, guarantees execution replicability. We designed automated unit tests to verify mathematical boundaries (e.g., ensuring CDF probabilities never exceed 1.0) and to confirm that our C++ quantile estimators accurately recover parameters from known synthetic seeds. During the peer-review phase, the provided test plan allowed independent validation of these workflows.

4. Self-Reflection and Critique

The development of `BFRdist` was highly educational, bridging the gap between theoretical statistics and practical software engineering. However, the process revealed several areas where our initial approach could have been improved.

4.1 Challenges Encountered

Our most significant technical hurdle was the `plot.bfr()` S3 method, which replicates the novel $SK_G \times KU_M$ distributional diagram. Generating the parametric bootstrap distributions and drawing the 90% and 95% confidence ellipses required complex covariance matrix calculations. Early iterations of the plot function crashed when empirical datasets lacked sufficient variance, resulting in singular covariance matrices. We had to retroactively implement robust error handling and fallback logic to gracefully handle degenerate data.

Furthermore, debugging the C++ implementation of the Type 7 quantile algorithm was challenging. Translating R's 1-based indexing logic to C++'s 0-based indexing caused off-by-one errors in early builds, which were only caught after expanding our `testthat` suite to handle very small sample sizes.

4.2 What We Would Do Differently

If we were to restart this project from scratch, we would alter our development methodology in the following ways: - **Test-Driven Development (TDD):** We initially wrote the core mathematical functions and only added `testthat` scripts prior to the draft package submission. In retrospect, writing the boundary tests (e.g., testing data inputs containing 0 or 1 exactly) before writing the functions would have prevented several silent mathematical failures early in the process. - **Expanding the C++ Backend:** Currently, only the Quantile Estimator (QE) utilizes `Rcpp`. The Maximum Likelihood Estimator (MLE) relies on R's native `optim()` function. Given the computational intensity of estimating parameters for massive remote sensing datasets, rewriting the MLE log-likelihood functions in C++ would vastly improve the package's overall execution speed. - **Handling the $\beta < 1$ Regime:** The theoretical paper [@bhattacharya2024] notes that the BFR distribution becomes U-shaped when $\beta < 1$. We did not sufficiently isolate and test this regime during our initial data validation phase. Future iterations should include specific warnings or distinct initial parameter guesses in the MLE loop when empirical data exhibits extreme bimodality at the boundaries.

5. Conclusion

The `BFRdist` package successfully encapsulates a complex, novel statistical framework into a user-friendly, object-oriented R interface. By leveraging C++ for computationally intensive quantile estimation and implementing robust S3 methods for visualization, the package serves as a practical tool for environmental data analysis. The rigorous application of unit testing and adherence to reproducibility standards ensure the software is reliable and extensible. The challenges faced during development, particularly regarding numerical stability and bootstrap confidence intervals, provided invaluable experience in advanced software engineering for data science.

References

- Bhattacharya, A., Frery, A. C., & Rey, A. A. (2026). A Statistical Framework for Bounded Measurements with Applications to SAR Data. *IEEE Transactions on Geoscience and Remote Sensing*.
- Frery, A. C., Gomez, L., & Medeiros, A. C. (2020). A Badging System for Reproducibility and Replicability in Remote Sensing Research. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 13, 4988-4995.